Cartographie 3D du campus avec un télémètre laser SMART-UHA

Halim BALA[1], Anas IBRAHIM[1]

[1]12 rue des frères Lumière 68039, Mulhouse
Université de Haute Alsace
Ecole Nationale Supérieure d'Ingénieurs Sud Alsace (ENSISA)
halim.bala@uha.fr, anas-ahmed-elsaeed.ibrahim@uha.fr

Résumé— **Le projet "Cartographie 3D du campus avec un télémètre laser" vise à permettre au robot SMART-UHA de naviguer de manière autonome tout en se localisant et en cartographiant un environnement inconnu. Ce travail s'appuie sur des recherches précédentes ayant implémenté l'algorithme SLAM LeGO-LOAM. Notre étude approfondit cet algorithme en évaluant ses performances et en comparant ses résultats à ceux d'un autre algorithme, FLOAM, reconnu pour sa rapidité et sa précision. Une série d'expériences a été réalisée pour comparer les cartes et trajectoires générées. Cette analyse met en lumière les atouts et limites des deux approches et propose des pistes d'amélioration pour la navigation autonome du robot SMART-UHA.**

Mots-clés— **SLAM, LeGO-LOAM, FLOAM, Cartographie 3D, Localisation, Robot autonome, LIDAR, Navigation autonome.**

I. INTRODUCTION

Le projet intitulé **"Cartographie 3D du campus avec un télémètre laser SMART-UHA"** a pour objectif principal de permettre au robot SMART-UHA de naviguer de manière autonome dans un environnement inconnu, tout en se localisant et en cartographiant son environnement en temps réel. Le projet actuel s'inscrit dans la continuité des travaux menés l'année dernière **[1]** dans le cadre d'un projet intitulé « Comment trouver mon chemin ? ».

Ces étudiantes **[1]** ont réussi à implémenter un algorithme SLAM (Simultaneous Localization And Mapping), permettant au robot SMART-UHA de naviguer de manière autonome tout en se localisant et en cartographiant son environnement.

L'objectif principal de notre travail est d'approfondir l'étude de l'algorithme SLAM qu'elles ont mis en œuvre, afin d'en comprendre les forces et les limites. Dans un premier temps, nous avons analysé les résultats obtenus par l'algorithme LeGO_LOAM (Lightweigth and Ground-Optimized LOAM) qu'elles ont utilisé. Sur la base des observations tirées de cette analyse **[1] [2]**, nous avons choisi d'implémenter un autre algorithme SLAM, nommé FLOAM (Fast LIDAR Odometry and Mapping), reconnu pour ses performances supérieures en termes de rapidité et de précision **[3]**.

Une fois cette implémentation achevée, nous avons entrepris une série d'expériences comparatives entre les deux algorithmes, en évaluant leurs performances respectives en matière de localisation et de cartographie. Ces expériences ont permis de mesurer la précision des cartes générées et des trajectoires estimées par chacun des algorithmes.

Cette étude comparative vise à mettre en lumière les atouts et les limites des algorithmes LeGO-LOAM et FLOAM, tout proposant des perspectives d'amélioration pour le robot SMART-UHA dans des environnements complexe.

II. MATERIEL

Dans le cadre de notre projet, plusieurs équipements ont été utilisés afin de mettre en pratique de valider les concepts étudiés en théorie. Ces matériels jouent un rôle essentiel dans la mise en œuvre des algorithmes SLAM et dans l'acquisition de données pour la navigation et la cartographie du robot SMART-UHA

A. Robot SMART-UHA

Construit par Kompai, ce projet a été réalisé en association avec l'Institut de Recherche en Informatique, Mathématiques, Automatique et Signal (IRIMAS) et l'Ecole Nationale Supérieure d'Ingénieurs Sud-Alsace (ENSISA) de l'université de Haute-Alsace **[4]**.

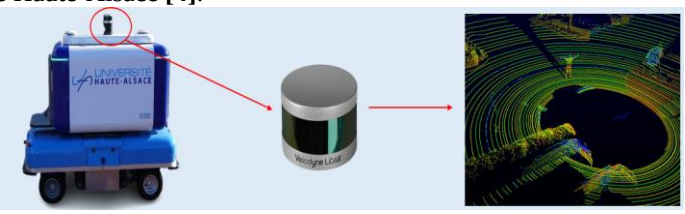


Fig. 1. Robot smart-sha.

La figure illustre le processus d'acquisition de données environnementales à l'aide du capteur LiDAR monté sur le robot Smart_uha. Après avoir collecté les données, l'environnement ROS est utilisé pour leur visualisation, permettant une analyse détaillée de l'espace exploré par le robot.

Ce robot est équipé pour **[4]** :

- La détection des obstacles.
- La livraison et la réception à l'aide d'une application mobile.
- La navigation autonome.
- La recharge électrique automatique.

B. Capteur LIDAR

Sur le robot SMART-UHA, le capteur LIDAR utilisé appartient à la famille Velodyne VLP 16. Ce capteur génère un nuage de points 3D représentant l'environnement autour du robot et il mesure la distance entre le capteur et les objets

environnants à l'aide de faisceaux laser, tout en capturant la réflectance des surfaces **[5]**.

- Le Velodyne VLP 16 possède 16 lasers verticaux répartis entre -15 degrés et 15 degrés.
- Il effectue une rotation pour scanner une vue horizontale à 360 degrés.
- Il renvoie la distance, l'angle vertical et l'angle horizontale de chaque point scanné.

C. ROS (Robot Operating System)

ROS est un ensemble de bibliothèques logicielles et d'outils open source conçus pour aider au développement d'applications robotiques. Il fournit des services attendus d'un système d'exploitation, tels que le contrôle des dispositifs bas niveau, l'implémentation de fonctionnalités couramment utilisées, la communication inter-processus et la gestion de package **[8]**.

Dans ce projet, le Robot Operating System (ROS) a été utilisé comme plateforme logicielle pour la gestion et l'analyse des données provenant du capteur LIDAR monté sur le robot SMART-UHA.

Grâce à ses outils performants tels que RViz, nous avons pu visualiser en temps réel les nuages de point 3D générés par le capteur Velodyne VLP 16.

III. SLAM (SIMULTANEOUS LOCALIZATION AND MAPPING)

Le SLAM est une technique clé en robotique permettant à un robot ou un système mobile de construire une carte d'un environnement inconnu tout en déterminant simultanément sa position au sein de cet environnement. Cela est réalisé à l'aide de capteurs, tels que des LIDARs, des Caméras ou d'autres dispositifs, qui fournissent des données sur les caractéristiques de l'environnement **[6]**.

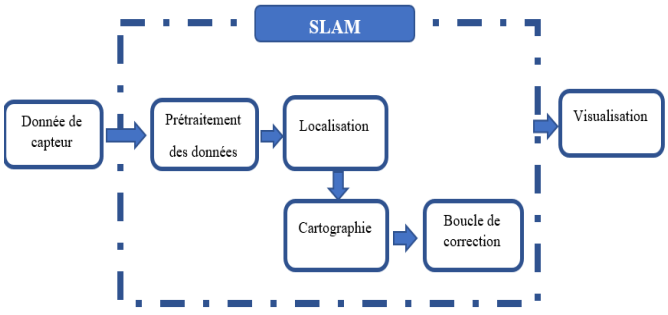


Fig. 2. Diagramme du processus général du SLAM, cette figure illustre les étapes principales pour la localisation et la cartographie.

Le processus général du SLAM peut être décomposé en plusieurs étapes principales :

A. Collecte des données de capteur

Les capteurs, tels que le LIDAR Velodyne VLP 16, génèrent des données brutes (comme un nuage de points 3D) sur l'environnement. Ces données constituent l'entrée principale pour le SLAM.

B. Prétraitement des données

Les informations brutes sont filtrées et transformées pour extraire des caractéristiques utiles, réduire le bruit et structurer les données de manière exploitable par les algorithmes.

C. Localisation

Le SLAM utilise les données prétraitées pour estimer la position et l'orientation actuelles du robot.

D. Cartographie

Simultanément, les informations environnementales recueillies sont utilisées pour créer une carte, souvent sous forme de représentation 2D ou 3D.

E. Boucle de correction

Une fois que le robot détecte qu'il est passé par un endroit déjà visité (phénomène de fermeture de boucle), le SLAM ajuste la carte et la localisation en conséquence pour minimiser les erreurs accumulées.

IV. ETAT DE L'ART

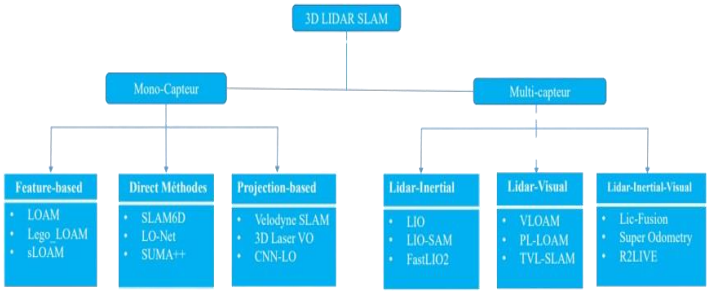


Fig. 3. Différents algorithmes de SLAM : cette figure illustre les différents algorithmes SLAM. Certains fonctionnent avec un seul capteur, tandis que d'autres peuvent utiliser plusieurs capteurs **[5].**

Le SLAM a connu des progrès significatifs grâce à l'amélioration des capteurs, des techniques d'optimisation et de la puissance de calcul. Cependant, cette évolution a conduit au développement de multiples algorithmes SLAM, chacun optimisé pour des cas d'utilisation spécifiques, comme les environnements intérieurs, les applications en temps réel ou les terrains accidentés.

Les variations entre ces algorithmes découlent de plusieurs facteurs :

- Types de capteur utilisé.
- Contraintes Matérielles.
- Types d'environnements.
- Objectifs de performance.

A. L'algorithme LOAM et ses variantes

L'algorithme LOAM (Lidar Odometry and Mapping), introduit en 2014 par Zhang et Singh de l'institut robotique de l'université Carnegie Mellon **[7]**, a marqué une étape importante dans le domaine du SLAM LiDAR grâce à sa précision et à son efficacité.

Cependant, au fil du temps, plusieurs variantes comme LeGO-LOAM, Fast LOAM et A-LOAM ont émergé pour répondre à des besoins spécifiques et surmonter certaines limites de LOAM **[7]**.

B. L'algorithme LeGO- LOAM

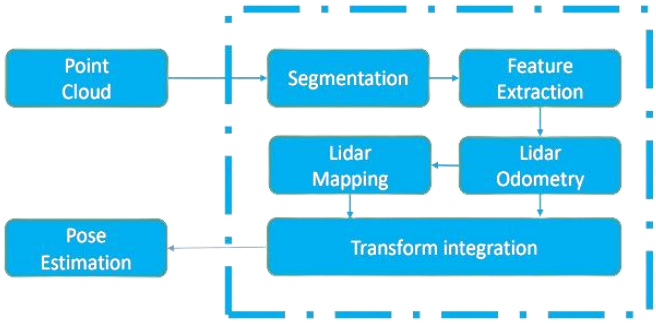


Fig. 4. Schéma fonctionnel de l'algorithme LeGO-LOAM.

LeGO-LOAM (Lightweight and Ground-Optimized LiDAR Odometry and Mapping) est une méthode optimisée pour estimer en temps réel la position d'un véhicule au sol en utilisant un capteur LiDAR. Voici comment il fonctionne et ses caractéristiques principales **[2]** :

- Segmentation des nuages de points : Le premier pas est d'appliquer une segmentation aux données des nuages de points pour éliminer le bruit et conserver les caractéristiques importantes. Cela inclut l'identification des caractéristiques planes et des contours (edge features).
- Extraction des caractéristiques : Des caractéristiques distinctives, comme les points sur des plans ou des arêtes, sont extraites à partir des nuages de points. Ces caractéristiques sont utilisées pour définir des correspondances entre les scans consécutifs.
- Optimisation de Levenberg-Marquardt : Une méthode d'optimisation en deux étapes est utilisée pour estimer les transformations (rotation et translation) entre les scans successifs. Les points extraits des arêtes et des plans servent de base pour résoudre les composantes de la transformation.
- Intégration des transformations : Les transformations estimées sont intégrées dans une carte globale. Le processus inclut la mise à jour de la position du véhicule (odométrie) et de la carte environnante.

C. L'algorithme FLOAM

L'algorithme FLOAM (Fast Lightweight and Odometry and Mapping) est une variante optimisée de l'algorithme LOAM (Lidar Odometry and Mapping). Il a été conçu pour répondre aux besoins croissants de rapidité et d'efficacité dans les systèmes SLAM (Simultaneous Localization and Mapping) utilisant des capteurs LIDAR **[3]**.

FLOAM vise à réduire le temps de calcul tout en maintenant une précision élevée dans la localisation et la cartographie, il fonction de la manière suivante **[3]** :

- Modèle de capteur et extraction des caractéristiques : Le capteur LiDAR collecte un nuage de points en scannant l'environnement. Les points caractéristiques sont extraits et classés en deux catégories : points sur les arêtes (edge features) et points sur les plans (planar features). Cette étape vise à simplifier le traitement en éliminant les points moins significatifs.
- Compensation des distorsions : Contrairement aux méthodes classiques qui utilisent des calculs itératifs coûteux, FLOAM applique une compensation en : a. Prédiction du mouvement en supposant une vitesse angulaire et linéaire constante. b. Ajustement des distorsions après l'estimation initiale de la pose pour améliorer la précision.
- Estimation de la pose : Les points caractéristiques (arêtes et plans) extraits du scan actuel sont alignés avec une carte globale (edge map et plane map).

  La pose du robot est optimisée en minimisant les distances entre les points locaux et les entités globales (plans ou arêtes).
- Construction de la carte globale : La carte globale est mise à jour de manière incrémentale en utilisant des images clés (keyframes), sélectionnées selon des seuils de déplacement et de rotation. Une optimisation supplémentaire est appliquée pour corriger les distorsions des scans, et un échantillonnage en voxelisation est utilisé pour réduire la taille de la carte.

D. Performance des algorithmes

TABLE I
TABLEAU COMPARATIVE DES PERFORMANCES DES ALGORITHMES
VOIR LES REFEFRENCES SUIVANTES **[2]**, **[3]**, **[7]** ET **[9]**

Critères	LOAM	LeGO-LOAM	FLOAM
Erreur de localisation translatoire (KITTI)	1.14	0.4	0.45
Erreur de localisation rotationnelle (KITTI)	0.82	1.14	0.49
Temps de calcul moyen (ms)	1.69	0.39	0.43
Usage moyen de CPU (%)	6.637%	4.433%	Très optimisé (< LeGO-LOAM et LOAM)
Extraction des caractéristique	Génère moins de caractéristique exploitable	Meilleure extraction des caractéristique grâce à la segmentation	Performance optimisée en extraction rapide

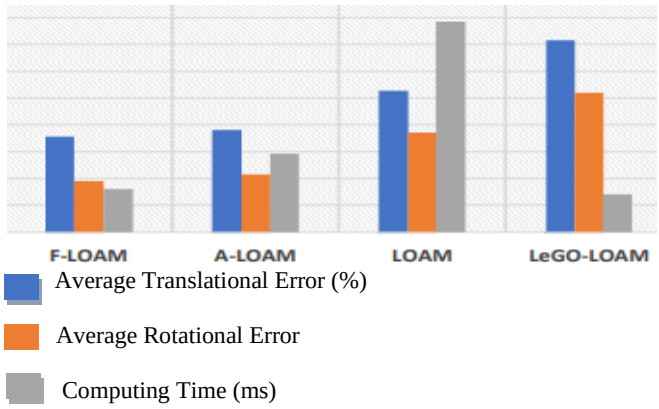


Fig. 5. Comparaison des différentes approches de localisation sur la séquence du jeu de données KITTI. Cette figure illustre les performances de chaque algorithme, en mettant en évidence leurs avantages et leurs limitations.

- F-LOAM est le meilleur algorithme en termes de précision et d'efficacité, avec des performances supérieures à celles de LOAM et LeGO-LOAM sur tous les paramètres testés.
- LeGO-LOAM est une bonne alternative lorsque les performances en temps réel sont importantes mais avec une précision légèrement inférieure.
- LOAM, bien que précis, est moins performant en termes de temps de calcul, ce qui le rend moins adapté aux applications temps réel.

V. Methodologie et travail experimental

Dans cette section, nous avons mené une série d'expériences sur deux algorithmes SLAM afin d'évaluer leurs performances et de déterminer celui qui serait le plus adapté pour permettre à un robot de naviguer de manière autonome tout en se localisant et en cartographiant un environnement inconnu.

Nous avons commencé par réimplémenter l'algorithme SLAM intitulé LeGO-LOAM (Lightweight and Ground-Optimized LOAM), déjà utilisé dans un projet précédent intitulé "Comment trouver mon chemin ?". Après des recherches approfondies sur cet algorithme et son fonctionnement, nous avons validé les observations précédentes concernant sa qualité en termes de localisation et de cartographie. Ces résultats ont confirmé ses performances, mais nous ont également permis d'identifier certaines limites, nous orientant vers la recherche d'un algorithme offrant une précision et une efficacité plus intéressante.

Nous avons alors choisi de travailler avec l'algorithme FLOAM (Fast LiDAR Odometry and Mapping), qui présente plusieurs avantages **[3]** :

- Rapidité : Grâce à son approche non itérative pour la compensation des distorsions, il réduit considérablement les coûts computationnels.
- Précision : Il offre une localisation et une cartographie de grande fiabilité en optimisant les caractéristiques locales extraites des données du LiDAR.
- Robustesse : Très robuste dans des environnements variés, qu'ils soient statiques ou dynamiques, comme des entrepôts ou des environnements urbains.

Ces expériences nous ont permis d'analyser en détail les performances respectives des deux algorithmes, afin de choisir la solution la plus adaptée à l'intégration sur un robot autonome.

A. Test et résultat sur l'algorithme LeGO-LOAM

Pour évaluer les performances de l'algorithme LeGO-LOAM, une expérience a été réalisée dans le couloir de l'institut. Le parcours consistait à démarrer d'un point de repère A, parcourir un trajet le long du couloir, puis revenir au point de départ.

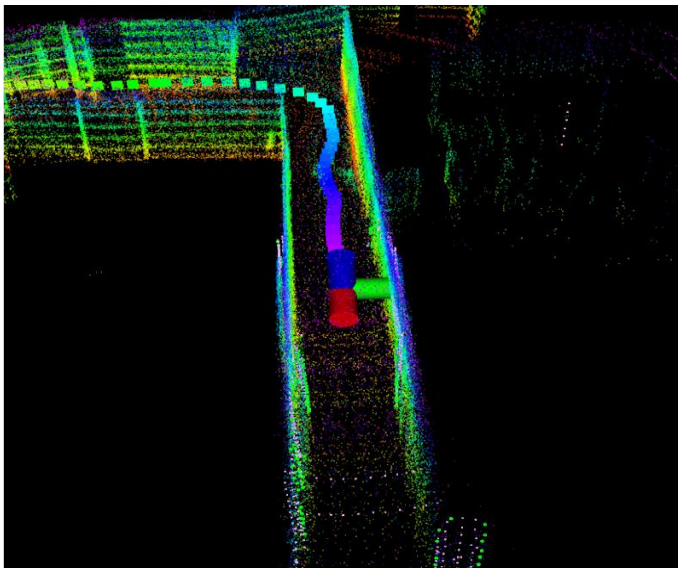


Fig. 6. Localisation et cartographie avec l'algorithme slam LeGO-LOAM, cette image est extraite de l'expérience réalisé dans le bâtiment de l'ENSISA.

L'algorithme LeGO-LOAM a montré une capacité remarquable à reconnaître les objets environnants, tels que les murs, le sol, et même les humains présents dans l'environnement. La cartographie générée était particulièrement détaillée, offrant une visualisation riche des structures environnantes sous forme de nuages de points.

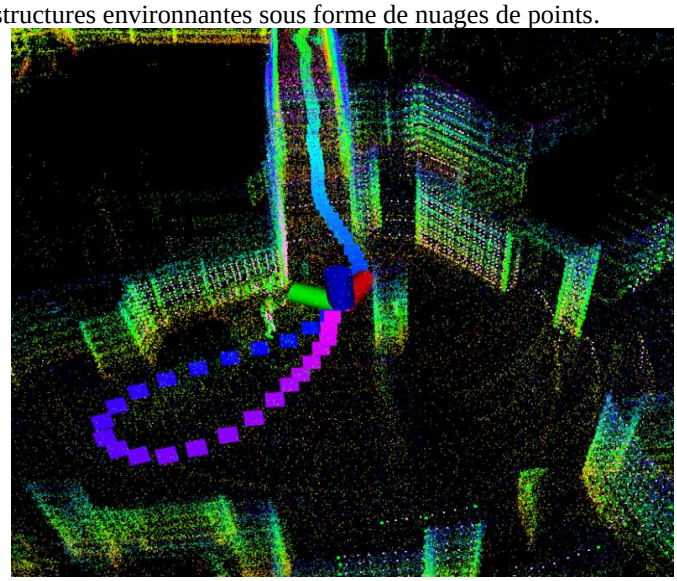


Fig. 7. Localisation et cartographie avec l'algorithme slam LeGO-LOAM

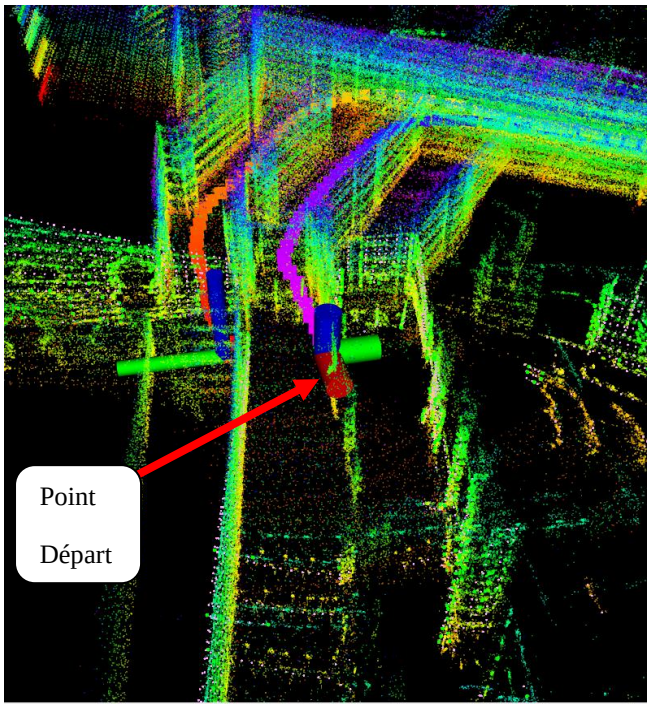


Fig. 8. Localisation et cartographie avec l'algorithme slam LeGO-LOAM sur le point de départ. Le parcourt est de démarrer d'un point ensuite faire u n tour et revenir sur le point de départ.

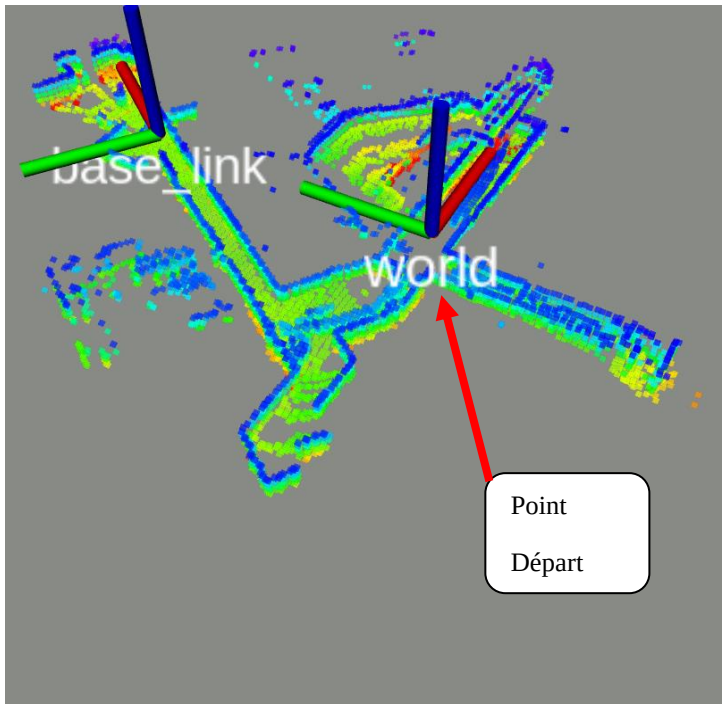


Fig. 9. Localisation et cartographie avec l'algorithme slam FLOAM sur le point de départ. Le parcourt est de démarrer d'un point ensuite faire un tour et revenir sur le point de départ.

Cependant, des limites ont été identifiées lors du retour vers le point de départ. Bien que l'algorithme ait correctement cartographié la majorité du trajet, une erreur de localisation a été observée. En effet, au lieu d'indiquer qu'il était revenu au point initial, LeGO-LOAM a confondu le couloir contenant le point de départ avec un autre. Cela a conduit à une cartographie erronée où deux couloirs distincts ont été représentés, alors qu'il n'en existe qu'un seul.

Cette erreur souligne une faiblesse de l'algorithme dans les situations où une localisation précise au point de départ est essentielle notamment dans un environnement intérieur. Les erreurs observées pourraient être attribuées à la surcharge des détails dans les nuages de points ou à un manque de robustesse face aux environnements similaires.

La densité des données générées par le capteur semble avoir compliqué l'identification précise du point de départ et la correspondance des caractéristiques environnementales.

B. Test et résultat sur l'algorithme FLOAM

Dans cette section, nous évaluons les performances de l'algorithme FLOAM. Une expérience similaire à celle réalisée avec LeGO-LOAM a été conduite. Le parcours consistait à partir d'un point de repère, suivre un itinéraire à travers le couloir de l'institut, puis revenir au point de départ.

Contrairement à LeGO-LOAM, FLOAM génère une cartographie avec moins de détails. Par exemple, les murs du bâtiment ENSISA n'apparaissent pas clairement dans la reconstruction, ce qui montre une limitation dans la richesse des données environnementales fournies.

Cependant, malgré cette réduction des détails, FLOAM s'est distingué par sa précision en localisation. Lors du retour, l'algorithme a correctement identifié le point de départ, sans confusion ni erreur dans la cartographie. Cela démontre une meilleure robustesse pour la localisation, même si la cartographie est moins détaillée.

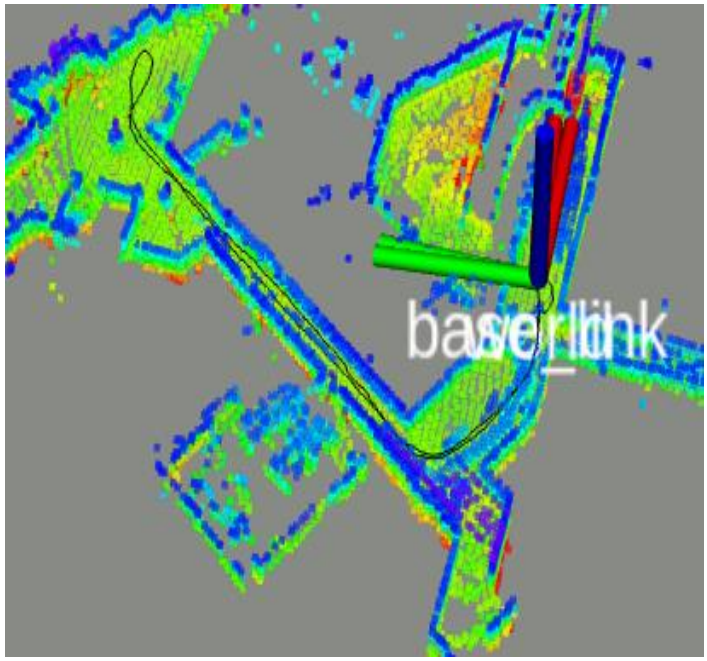


Fig. 10. Revenir sur le point de départ avec l'algorithme FLOAM.

L'un des éléments clés de cette précision est la réduction du niveau de détail dans les nuages de points générés. Contrairement à LeGO-LOAM, FLOAM sacrifie une partie des informations sur l'environnement (par exemple, les murs et autres structures) pour se concentrer sur la trajectoire et la cohérence globale du parcours. Ce compromis permet à l'algorithme de réduire les risques de confusion dans des environnements fortement chargés en détails, comme des couloirs étroits ou encombrés.

En pratique, cette précision fait de FLOAM un candidat idéal pour des applications où la localisation est prioritaire par rapport à la cartographie, comme la navigation autonome ou la robotique industrielle. FLOAM montre ainsi une robustesse qui pourrait répondre aux besoins d'applications exigeant une trajectoire fiable avec un faible coût de calcul.

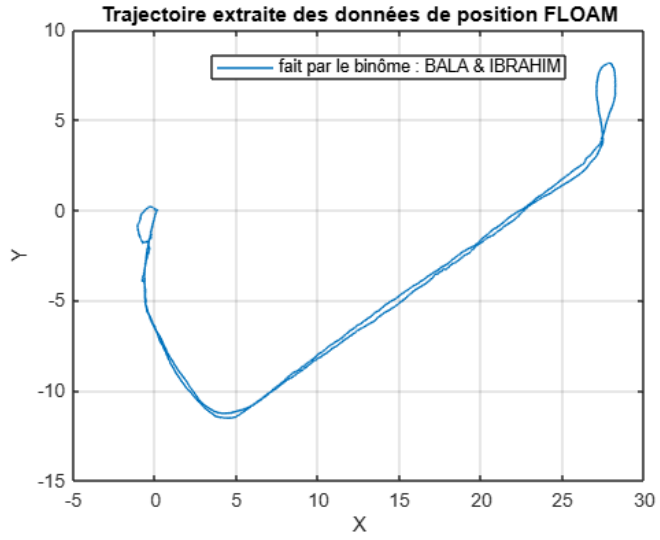


Fig. 11. Représentation du parcours effectué avec l'algorithme FLOAM sur MATLAB.

Les résultats démontrent que FLOAM est particulièrement efficace pour éviter les erreurs de localisation observées avec d'autres algorithmes, comme LeGO-LOAM.

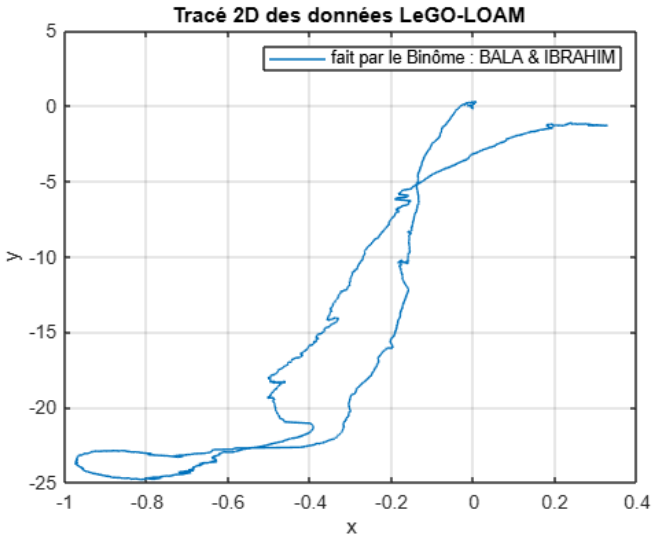


Fig. 12. Représentation du parcours effectué avec l'algorithme LeGO_LOAM sur MATLAB.

Cette performance, combinée à sa simplicité, illustre clairement pourquoi FLOAM est un choix performant pour les environnements contrôlés ou semi-structurés.

VI. CONCLUSION

Dans le projet "Cartographie 3D du campus avec un télémètre laser SMART-UHA", nous avons étudié et comparé les performances des algorithmes SLAM, notamment Lego-LOAM et FLOAM. Les expériences menées ont montré que FLOAM est plus précis, surtout dans des environnements semi-structurés, en retrouvant avec exactitude le point de départ et en générant des cartes exploitables, bien que moins détaillées. Ces résultats confirment son efficacité pour des applications où la précision est cruciale, comme la navigation autonome. Ce travail souligne l'importance de choisir le bon algorithme en fonction des besoins spécifiques des applications.

Pour les travaux futurs, il sera nécessaire d'implémenter l'algorithme FLOAM sur le robot SMART-UHA. Il faudra également comparer les résultats des trajectoires de Lego-LOAM et FLOAM avec les données GPS. Enfin, des tests dans un environnement plus grand, tel que le campus de l'université, permettront d'évaluer la robustesse de l'algorithme sur des distances et des configurations plus complexes.

REFERENCES

[1] AGOURRAM Amina, GHATOUSS Doha. (2024). Comment trouver mon chemin ? Projet de recherche, Université de Haute-Alsace.

[2] Shan, T., & Englot, B. (2018). "LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping on Variable Terrain." 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS).

[3] Wang, H., Wang, C., Chen, C.-L., & Xie, L. (2021). FLOAM: Fast LiDAR Odometry and Mapping. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)

[4] université de haute-alsace (uha). Découvrez le robot autonome smart-uha. Disponible en ligne : https://www.uha.fr/fr/actualites-agenda-1/ca-s-estpasse-a-l-uha/decouvrez-le-robot-autonome-smartuha.html (consulte le 10 octobre 2024)

[5] RESEARCHGATE. vertical range of vlp-16. Disponible en ligne : https://www.researchgate.net/figure/vertical-rangeof-vlp-16_fig1_332686285 (consulte le 10 octobre 2024)

[6] Durrant-Whyte, h., & Bailey, t. (2006). "Simultaneous Localization And Mapping: PART I." ieee robotics & automation magazine, 13(2), 99–110

[7] Garigipati, B., Strokina, N., & Ghabcheloo, R. (2022). "evaluation and comparison of eight popular lidar and visual slam algorithms." arxiv preprint arxiv:2208.02063v1

[8] ROS wiki. Introduction to ROS. Disponible en ligne : https://wiki.ros.org/ros/introduction (consulte le 02 octobre 2024)

[9] Zhang, J., & Singh, S. (2014). "LOAM: Lidar Odometry And Mapping in real-time." proceedings of robotics: science and systems (RSS)